IE3142 · DEVOPS SECURITY

Secure Cloud Services and Serverless Architectures

Lecture 10 - From Cloud-Native Foundations to Serverless Security, Monitoring, and Incident Response

Ayesha Wijesooriya

LECTURE ROADMAP

Topics Covered

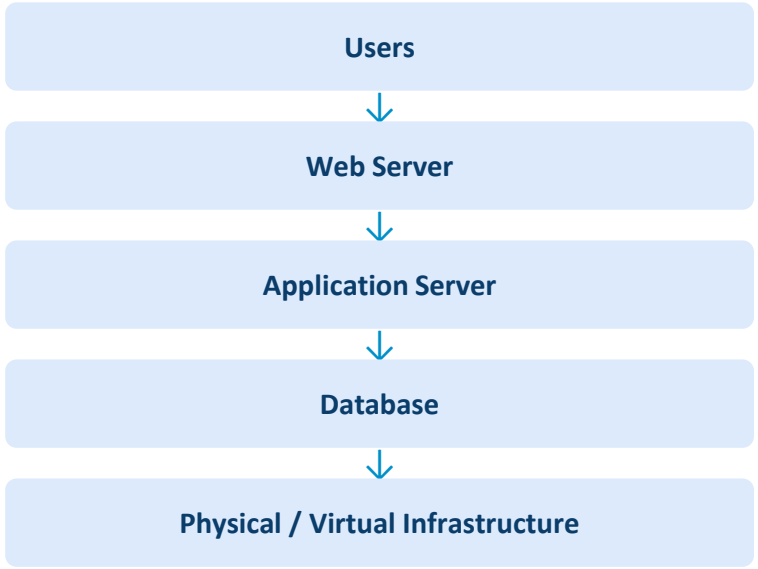
1. Cloud-Native Applications and Services
2. Introduction to Serverless Computing
3. Function as a Service (FaaS)
4. AWS Lambda
5. Azure Functions
6. Serverless Architecture
7. Serverless Security Threats
8. IAM and Least Privilege
9. Secure Functions and APIs
10. Secrets and Data Protection
11. Dependency and Supply-Chain Security
12. Serverless Security Best Practices
13. Security Monitoring and Logging
14. Incident Detection and Response

PART 1

Introduction to Cloud-Native and Serverless Computing

From Traditional to Cloud-Native Architecture

Traditional Architecture



Organization Manages

- Servers
- Operating systems
- Runtime environments
- Networking
- Scaling
- Patching

KEY IDEA

Cloud-native does not simply mean an application hosted in the cloud — it refers to applications designed to take advantage of cloud computing models and services.

Cloud-Native Capabilities

Modern cloud applications increasingly use cloud-native services designed to take advantage of:

1 Elastic Scaling
Resources grow and shrink automatically with demand.

2 Managed Services
Providers operate the underlying infrastructure.

3 Containers
Portable, consistent packaging for applications.

4 Microservices
Independently deployable application components.

5 APIs
Standardized interfaces connecting services.

6 Automation
Provisioning and operations without manual steps.

7 Infrastructure as Code
Infrastructure defined and versioned in code.

8 Serverless Computing
Execution without directly managing servers.

What are Cloud-Native Services?

Services designed and operated within cloud environments to provide compute, storage, databases, messaging, identity, monitoring, and application execution.

Compute

AWS Lambda · Azure Functions

Storage

Amazon S3 · Azure Blob Storage

Databases

Amazon DynamoDB · Azure Cosmos DB

Messaging / Events

Amazon EventBridge · SQS · Azure Service Bus · Event Grid

API Services

Amazon API Gateway · Azure API Management

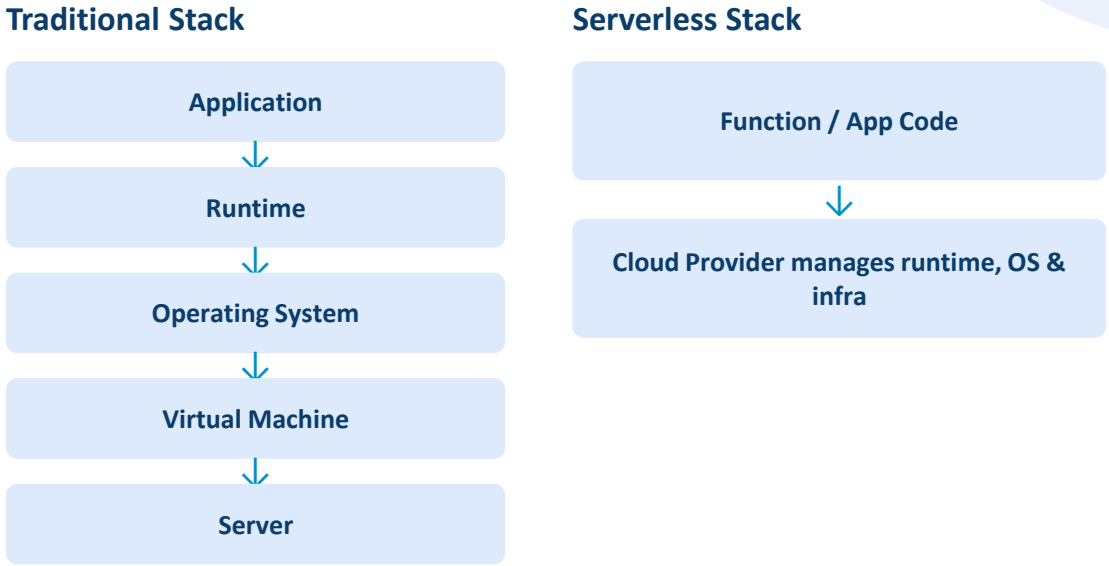
Cloud-native applications often combine several managed services instead of relying on one large server.

What is Serverless Computing?

A cloud computing execution model where the provider manages the underlying servers, operating systems, runtime infrastructure, availability, and much of the scaling required to execute an application.

Developers Focus On

- Application code
- Business logic
- Functions
- Data
- Permissions
- Application configuration



IMPORTANT POINT

Serverless does NOT mean there are no servers - developers and organizations simply do not directly provision or manage them.

Why Serverless Computing?

1 Automatic Scaling

Functions scale automatically with incoming requests or events.

2 Reduced Infrastructure Management

No need to directly manage servers or operating systems.

3 Pay-per-Use

Pay based on actual execution and resource consumption.

4 Faster Development

Developers focus more on application functionality.

5 High Availability

The provider manages underlying availability infrastructure.

6 Event-Driven Architecture

Functions execute automatically when events occur.

What is Function as a Service (FaaS)?

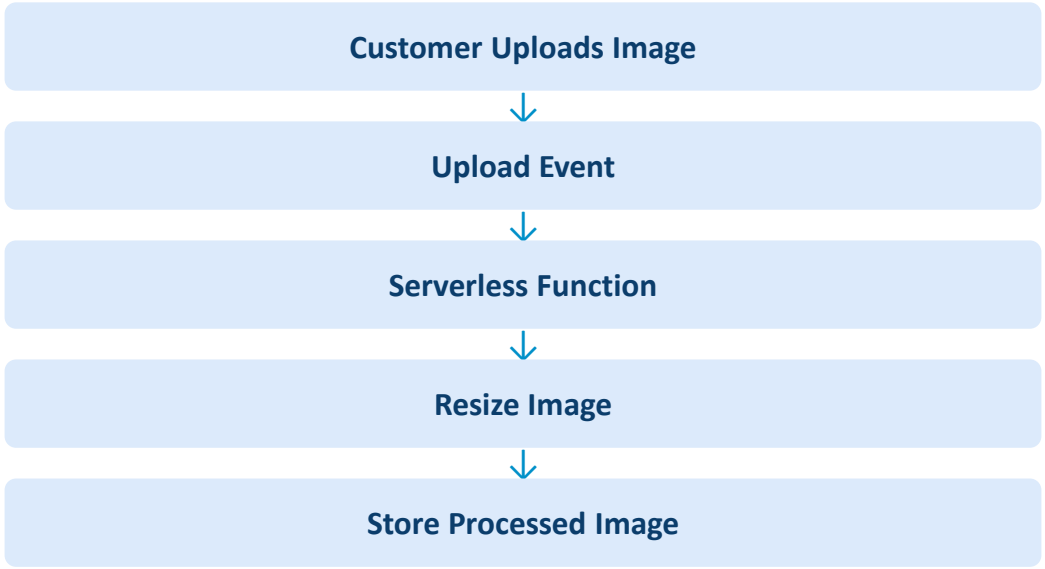
A serverless computing model where developers deploy individual functions that execute in response to specific events. A function normally performs a focused task.

Examples

- AWS Lambda
- Azure Functions
- Google Cloud Functions

The function runs when required and does not need a continuously running application server managed by the developer.

Example: Image Processing



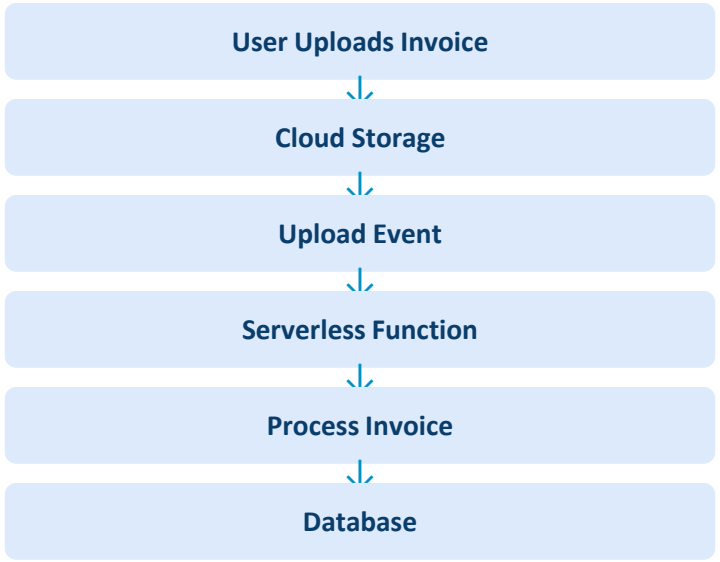
Event-Driven Serverless Architecture

Serverless applications are frequently event-driven — an event causes a function to execute.

Event Types

- HTTP / API request
- File upload
- Database change
- Message arriving in a queue
- Scheduled event
- User authentication event
- Cloud infrastructure event

Example: Invoice Upload



SECURITY IMPORTANCE

Every event source becomes part of the trust boundary and attack surface. Verify: who generated the event, is it authorized, can the input be trusted, and what permissions does the function have?

PART 2

Securing Cloud-Native Services

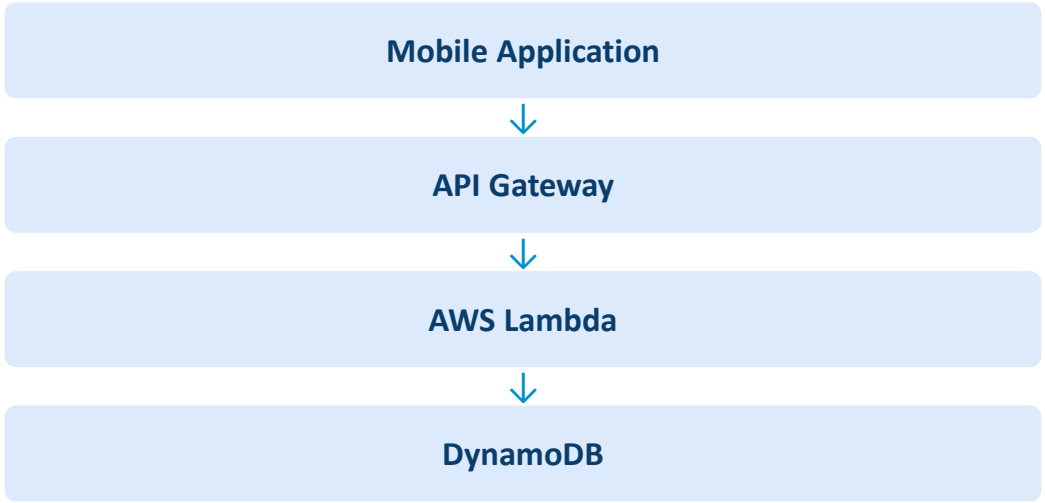
Introduction to AWS Lambda

A serverless compute service that executes code in response to events without requiring customers to provision or directly manage servers.

Triggered By

- API Gateway
- Amazon S3
- DynamoDB
- EventBridge
- SQS
- SNS

Example



The developer manages the function code and its permissions while AWS manages much of the execution infrastructure.

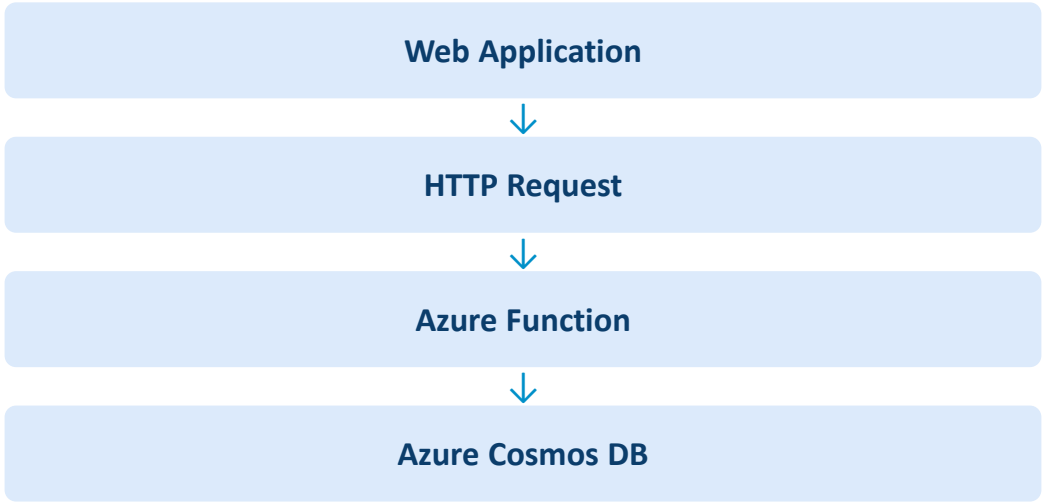
Introduction to Azure Functions

Microsoft's event-driven serverless compute service, allowing developers to concentrate on application logic while the cloud provider manages much of the underlying infrastructure.

Triggered By

- HTTP requests
- Timers
- Storage events
- Queue messages
- Event Grid events
- Service Bus messages

Example



SHARED RESPONSIBILITY

Traditional Server vs Serverless Security

IMPORTANT SECURITY PRINCIPLE

Moving to serverless removes some infrastructure management responsibilities but does not remove security responsibility.

Developers Still Need to Secure

- **Code**
- **IAM Permissions**
- **APIs**
- **Dependencies**
- **Secrets**
- **Data**
- **Event Sources**
- **Configuration**

Shared Responsibility in Serverless

Cloud Provider

- Physical infrastructure
- Hardware
- Host operating environment
- Managed execution infrastructure
- Platform availability

Customer

- Application code & function configuration
- Identity and permissions
- Data and secrets
- Dependencies
- API security & business logic
- Monitoring configuration

PART 3

Serverless Security Considerations

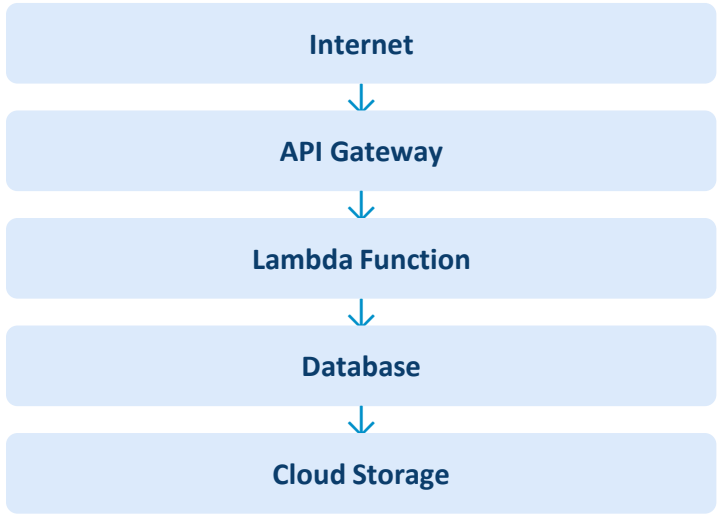
ATTACK SURFACE

Serverless Attack Surface

Serverless reduces some traditional infrastructure risks but introduces or emphasizes other attack surfaces.

- Function code
- API endpoints
- Event sources
- IAM roles
- Cloud storage
- Databases
- Third-party dependencies
- Environment variables
- Secrets
- CI/CD pipelines
- Cloud service configurations

Example Path



An attacker may target any component or the trust relationships between them.

ATTACK SURFACE

Common Serverless Security Risks

1 Insecure Function Code

Injection or insecure input handling.

2 Excessive Permissions

Functions with more access than necessary.

3 Insecure APIs

Weak authentication or authorization.

4 Vulnerable Dependencies

Packages with known vulnerabilities.

5 Exposed Secrets

Keys, credentials, or tokens stored insecurely.

6 Event Injection

Malicious input entering via event sources.

7 Security Misconfiguration

Incorrect cloud settings expose data.

8 Insufficient Logging

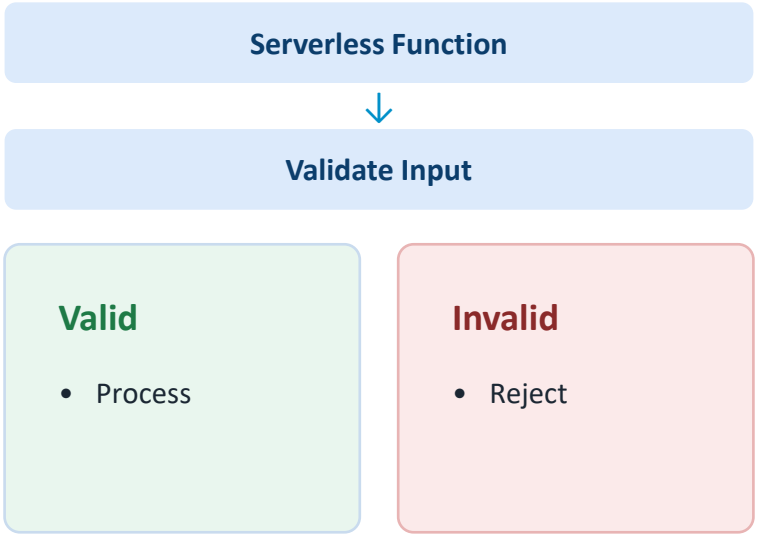
Attacks go undetected without monitoring.

Input Validation in Serverless Functions

Input Sources

- APIs
- Files
- Databases
- Message queues
- Cloud events
- Users
- External systems

Validation Flow



Secure Practices

- Validate type, length & format
- Apply allowlists where appropriate
- Sanitize or safely handle untrusted input
- Use parameterized database queries
- Reject unexpected data

Serverless does not eliminate traditional application vulnerabilities such as SQL injection.

IAM and Least Privilege in Serverless

Example

Lambda

↓

Read S3 Object

↓

Write DynamoDB Record

A function should not automatically receive AdministratorAccess.

PRINCIPLE OF LEAST PRIVILEGE

Grant each function only the permissions necessary for its intended task.

Benefits

- Limits damage if a function is compromised
- Reduces privilege escalation opportunities
- Reduces unauthorized access
- Limits lateral movement

IDENTITY & ACCESS

Function-Level Permissions

Three Functions

Upload Function

→ S3 Write

Payment Function

→ Payment Service

Report Function

→ Database Read

Instead of: Broad Shared Role

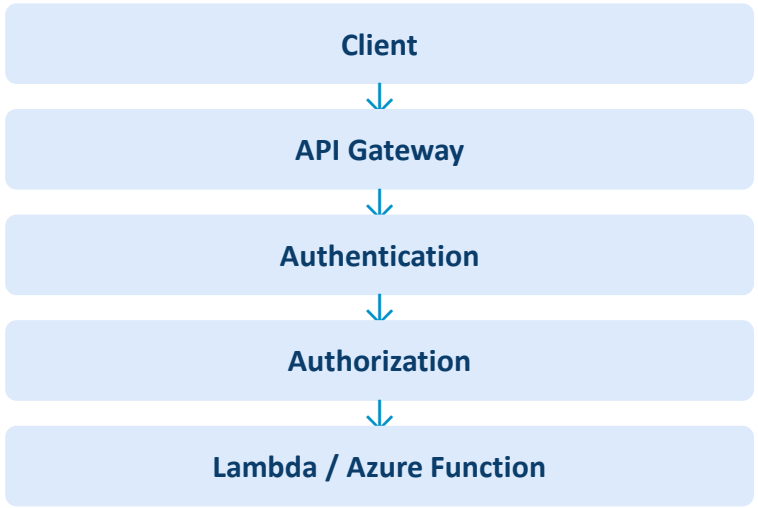
- All Functions → Administrator Role
- Every function can access every resource
- One compromised function risks everything

Separate, Scoped Permissions

- Upload Function → Storage permissions
- Payment Function → Payment permissions
- Report Function → Read-only database permissions
- Limits the blast radius if one function is compromised

Securing Serverless APIs

Request Flow



API Security Controls

- Strong authentication
- Proper authorization
- TLS / HTTPS
- Input validation
- Rate limiting
- Request size limits
- API logging
- Secure error handling
- Web Application Firewall where appropriate

IMPORTANT PRINCIPLE

Authentication asks: Who are you?

Authorization asks: Are you permitted to perform this action?

Both are required.

Secrets Management in Serverless

Functions often require database passwords, API keys, tokens, certificates, and encryption keys. These should not be hard-coded into source code.

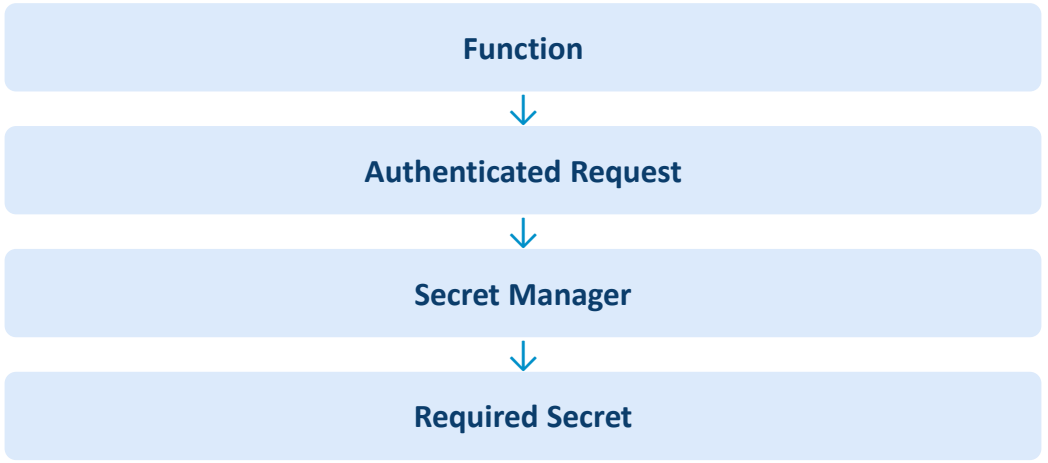
Insecure

```
API_KEY = "abcd123456"
DB_PASSWORD = "password123"
```

Better Approach

- AWS Secrets Manager
- AWS Systems Manager Parameter Store
- Azure Key Vault

Secret Retrieval Flow



Access to secrets should also follow least privilege.

Environment Variables and Sensitive Data

Environment variables are commonly used to provide configuration to serverless functions, e.g. DATABASE_HOST, API_ENDPOINT, REGION. Sensitive values require additional care.

Risks

- Accidental exposure through logs
- Excessive access permissions
- Debugging output
- Configuration leakage

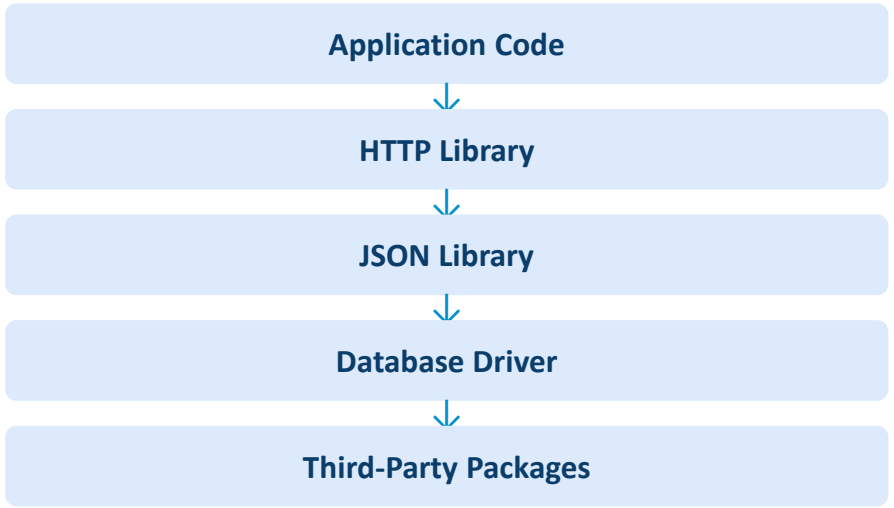
Best Practices

- Do not log secrets
- Restrict access to function configuration
- Encrypt sensitive configuration where supported
- Prefer dedicated secret-management systems for high-value secrets
- Rotate credentials where appropriate

SOFTWARE SUPPLY CHAIN

Dependency Security in Serverless

A Function's Dependencies



Best Practices

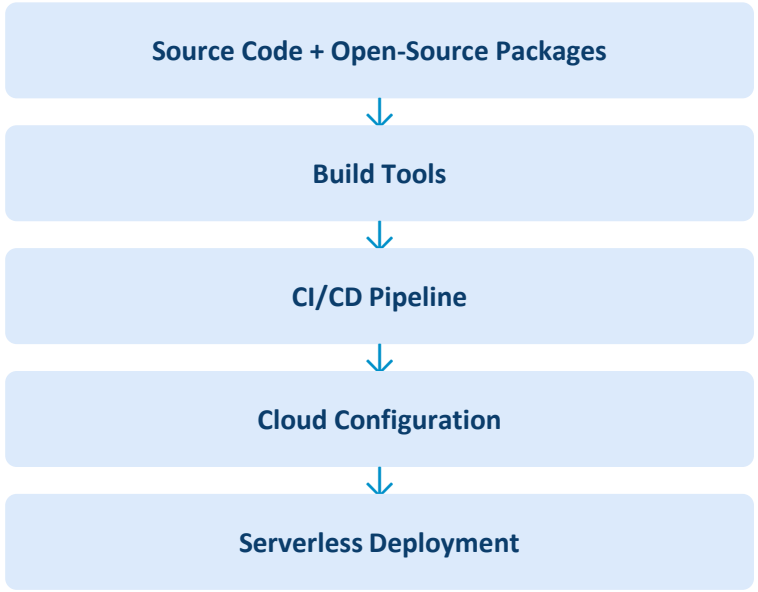
- Maintain dependency inventories
- Use Software Composition Analysis (SCA)
- Remove unused dependencies
- Update vulnerable packages
- Pin dependencies appropriately
- Generate an SBOM where required
- Use trusted package repositories

A vulnerability in one dependency can make the entire function vulnerable.

This connects directly to the SCA concept covered in the secure CI/CD lecture.

Serverless and Software Supply-Chain Security

Dependency Chain



AN ATTACKER COULD COMPROMISE

- Dependency
- Package repository
- Source-code repository
- Build pipeline
- Deployment credentials

An attacker does not necessarily need to attack the running function directly.

Security Controls

- SAST
- SCA
- Secret scanning
- Dependency pinning
- Artifact integrity verification
- Protected repositories
- Secure CI/CD credentials

Data Protection in Serverless Architectures

Serverless applications often process sensitive data across multiple managed services. Data should be protected across four dimensions:

1 At Rest

Databases, object storage, queues, and backups should be encrypted where appropriate.

2 In Transit

Use secure communication such as HTTPS and TLS.

3 Access

Use IAM, least privilege, resource policies, and strong authentication.

4 Data Minimization

Functions should access only the data they actually require.

Secure Error Handling

Serverless functions should not expose sensitive technical information through error messages.

Insecure Error

```
Database connection failed:
username=admin
host=db.internal.company
password=...
```

Better Error Handling

```
Unable to process the request.
```

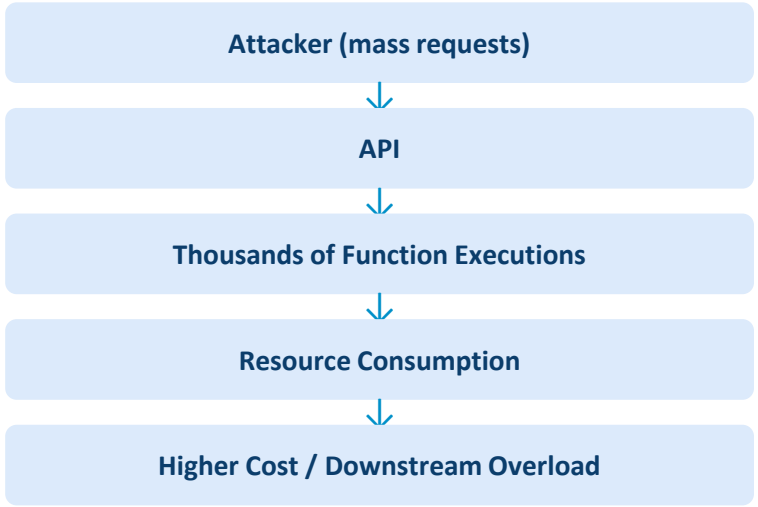
Internal logs can contain enough diagnostic information for investigation but should avoid recording passwords, tokens, or other secrets.

Best Practices

- Use generic user-facing errors
- Log appropriate diagnostic information
- Never log secrets
- Handle exceptions securely
- Avoid revealing internal architecture unnecessarily

Denial of Service and Resource Consumption

Attack Pattern



This May Cause

- Increased cloud costs
- Exhaustion of concurrency limits
- Overload of downstream databases/services
- Reduced availability

Controls

- Rate limiting
- API quotas
- Request throttling
- Authentication
- Budget / cost alerts
- Concurrency controls
- WAF / DDoS protections where appropriate

This risk is sometimes described as Denial of Wallet — malicious activity that primarily causes excessive cloud consumption and cost.

Function Timeout and Resource Limits

Functions Need Limits For

- Execution time
- Memory
- Concurrency
- Request size

Resource Limits Contribute To

- Availability
- Cost control
- Resilience
- Abuse prevention

Example

Expected

- Execution = 2 seconds

Abnormal

- Execution = 10 minutes

A configured timeout can terminate unexpectedly long execution before it causes wider impact.

BEST PRACTICES

Serverless Security Best Practices

Function Security

Validate inputs · secure coding · minimize permissions · handle errors securely

Identity

Least privilege · separate roles · avoid long-lived credentials

Secrets

Use managed secret storage · never hard-code secrets

Dependencies

Scan dependencies · patch vulnerable packages

Data

Encrypt sensitive data · restrict access

API

Authenticate & authorize requests · apply rate limiting

Monitoring

Enable logging · monitor unusual activity · configure alerts

PART 4

Serverless Security Monitoring

Why Monitor Serverless Applications?

Serverless infrastructure is highly dynamic. Traditional server monitoring techniques alone are therefore insufficient.

Functions May

- Start automatically
- Execute for milliseconds or seconds
- Scale rapidly
- Stop after completing work

Organizations Need Visibility Into

- Function invocation
- Authentication activity
- Errors & API requests
- IAM changes
- Configuration changes
- Data access
- Unusual execution patterns

MONITORING

What Should We Monitor?

Monitoring should establish what normal behavior looks like, so that unusual behavior can be identified.

MONITORING

Serverless Logging

A Log May Include

- Timestamp
- Function identifier
- Request/event identifier
- Authentication result
- Action performed
- Error status
- Relevant security context

Example Log Entry

```
2026-09-22 10:22:05
Function: PaymentProcessor
User: customer-102
Action: ProcessPayment
Result: SUCCESS
Request-ID: ABC123
```

AVOID LOGGING

- Passwords
- Access tokens
- Private keys
- Full payment-card data
- Sensitive personal information unless explicitly required and properly protected

MONITORING

AWS Serverless Monitoring

1 Amazon CloudWatch

Logs, metrics, alarms, and function performance information.

2 AWS CloudTrail

Records AWS API and account activity for security auditing and investigation.

3 AWS X-Ray

Distributed tracing to understand requests across distributed applications.

4 Amazon GuardDuty

Threat detection using signals from supported AWS data sources and services.

Together, these services provide different types of visibility rather than serving exactly the same purpose.

MONITORING

Azure Serverless Monitoring

1 Azure Monitor

Collects and analyzes telemetry from Azure resources.

2 Application Insights

Application performance monitoring and request-level visibility, including for Azure Functions.

3 Microsoft Defender for Cloud

Security posture management and workload protection capabilities.

4 Microsoft Sentinel

SIEM/SOAR capabilities for centralized security analytics, investigation, and response.

5 Azure Activity Log

Records subscription-level Azure control-plane events.

Detecting Suspicious Serverless Behavior

Possible Indicators

- Sudden increase in function invocations
- Repeated authentication failures
- Function executing at unusual times
- Unexpected IAM changes
- Access to unusual cloud resources
- Unexpected outbound network connections
- Large data transfers
- Unusual API requests
- Unexpected configuration changes
- Abnormal cloud costs

Normal

- Payment Function → Payment Database

Abnormal

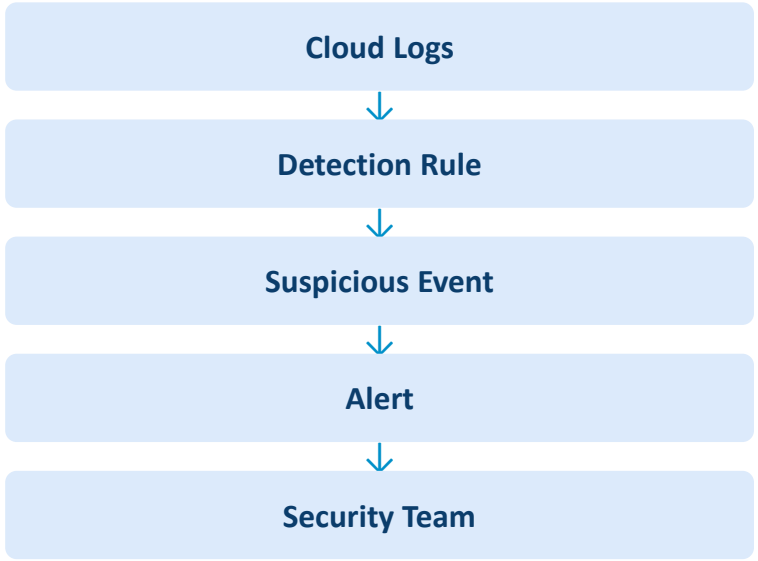
- Payment Function → Unknown External Host

This behavior should generate an investigation.

THREAT DETECTION

Alerts and Automated Detection

Detection Flow



Example Alert Conditions

- Administrator role assigned unexpectedly
- Function modified outside approved deployment process
- Large number of failed API requests
- Unusual secret access
- Unexpected spike in function execution
- Security configuration disabled

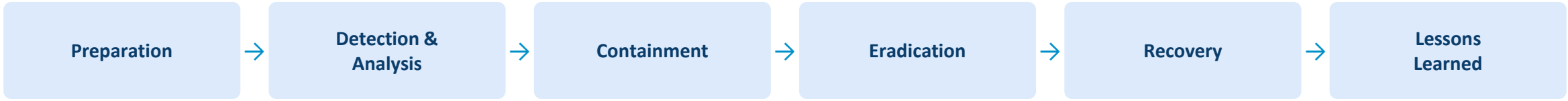
Alerts should be prioritized according to severity and risk to avoid overwhelming security teams.

PART 5

Serverless Incident Response

Incident Response in Serverless Environments

The Incident Response Lifecycle



Serverless environments change how evidence is collected and how containment is performed, since customers may not have direct access to the underlying server.

Investigation Relies Heavily On

- Cloud audit logs
- Function logs
- API logs
- IAM records
- Configuration history
- Application telemetry

Example Serverless Security Incident

Consider an AWS Lambda function used to process customer documents. An attacker obtains credentials with excessive permissions.

Attack Chain

Stolen Credentials

↓

Cloud Account Access

↓

Modify Lambda Function

↓

Malicious Code Added

↓

Customer Data Accessed

↓

Data Sent Externally

Potential Evidence

- CloudTrail events
- Lambda configuration history
- CloudWatch logs
- IAM activity
- API activity
- Storage-access logs
- Network telemetry where available

Detecting the Incident

Often First Detected Through

- Unexpected Lambda modification
- Unusual administrator activity
- Unexpected outbound communication
- Increased data access
- Security alert
- Abnormal API activity

INVESTIGATION QUESTIONS

Who modified the function?
When was it modified?
Which credentials were used?
What permissions did those credentials have?
What data did the function access?
Were other resources affected?
Was information exfiltrated?

Containment

The objective of containment is to stop the attack and prevent additional damage.

- Disable compromised credentials
- Revoke active sessions/tokens where supported
- Restrict IAM permissions
- Disable or isolate affected function/event source where appropriate
- Block malicious API access
- Rotate exposed secrets
- Restrict access to affected data
- Preserve relevant logs and evidence

Containment actions should be chosen carefully to balance security and service availability.

Eradication and Recovery

Eradication - Remove the Cause

- Remove malicious function code
- Patch vulnerable application code
- Remove malicious dependencies
- Correct IAM policies
- Replace compromised credentials
- Correct insecure cloud configurations

Recovery - Restore Trusted Services

- Redeploy known-good functions from an approved artifact
- Restore secure configuration
- Validate IAM permissions
- Re-enable event sources
- Increase monitoring temporarily
- Verify normal application behavior

Serverless Forensics

Traditional digital forensics may involve imaging physical disks or examining a persistent server. Serverless environments are different - the underlying execution infrastructure is managed by the cloud provider and function instances may be short-lived, so investigations rely heavily on cloud-native evidence.

Important Evidence May Include

- Function logs
- Cloud audit logs
- IAM logs
- API Gateway logs
- Storage access records
- Database audit logs
- Function configuration
- Deployment history
- Source repository history
- CI/CD pipeline logs
- Distributed traces

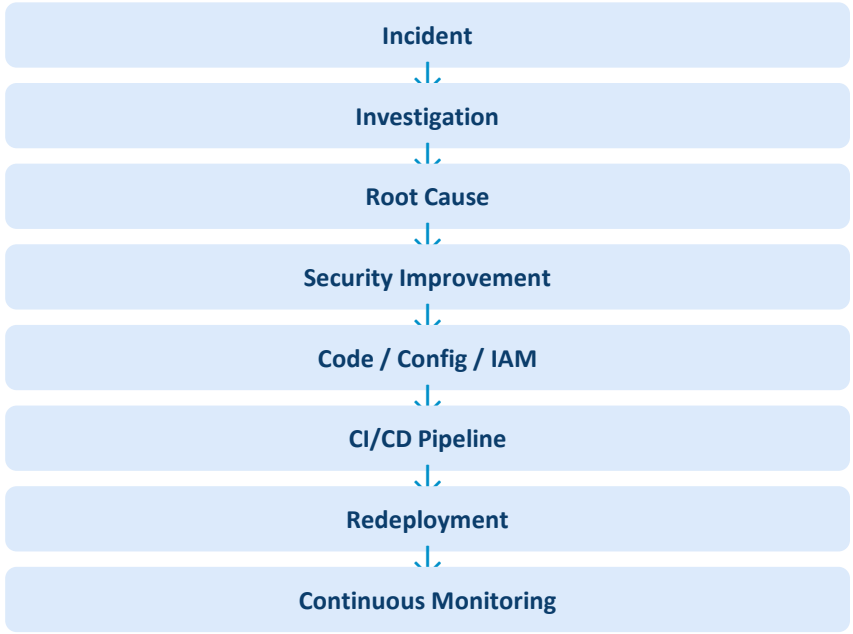
IMPORTANT PRINCIPLE

Logging and audit configuration must be established before an incident occurs. Evidence that was never collected may be impossible to reconstruct later.

DEVSECOPS FEEDBACK LOOP

Serverless Incident Response and DevSecOps

Feedback Loop



Examples

Incident: Vulnerable dependency exploited

→ **Improve SCA and dependency policies**

Incident: Excessive IAM permission abused

→ **Improve least-privilege policies**

Incident: Secret leaked

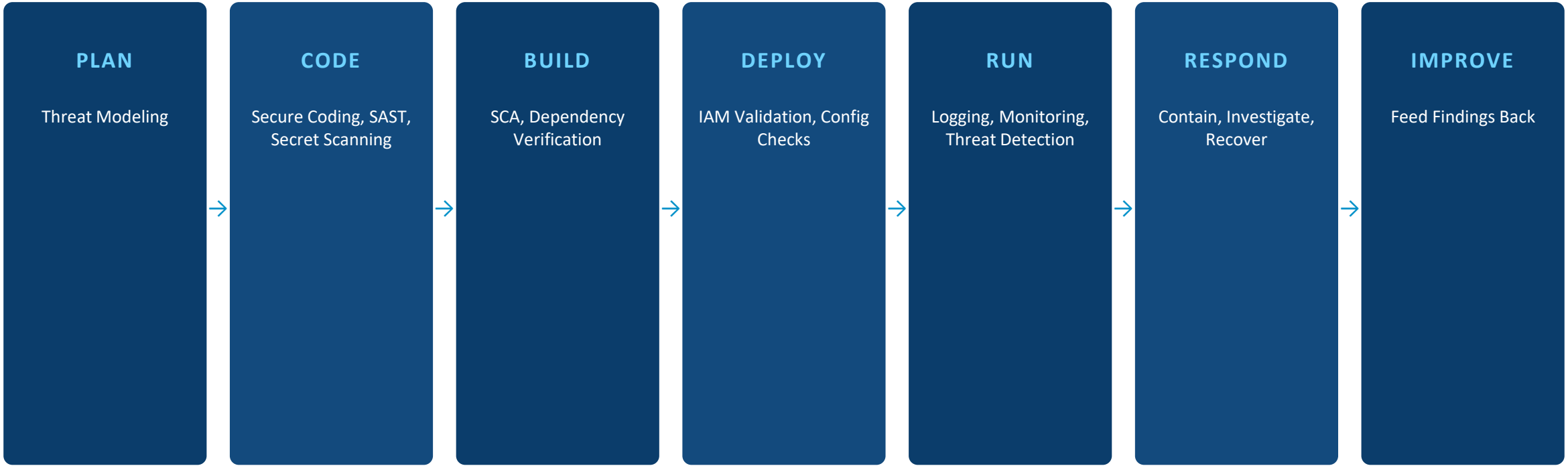
→ **Add secret scanning and improve secret management**

Incident: Malicious function modification

→ **Strengthen CI/CD deployment controls and audit alerts**

DEVSECOPS INTEGRATION

Serverless Security Across the Lifecycle



This is the direct connection between serverless security and DevSecOps.

PLATFORM COMPARISON

AWS Lambda vs Azure Functions: Security View

The names differ, but the security objectives remain similar:

Strong identity + least privilege + secure code +
protected secrets + secure APIs + monitoring + incident response

CHECKLIST

Serverless Security Best-Practice Checklist

Code
✓ Secure coding practices followed
✓ Input validated
✓ SAST completed

Dependencies
✓ SCA completed
✓ Vulnerable packages updated
✓ Unnecessary packages removed

IAM
✓ Least privilege applied
✓ Separate function roles used
✓ Excessive permissions removed

Secrets
✓ No hard-coded credentials
✓ Secret manager used

APIs
✓ Authentication enabled
✓ Authorization enforced
✓ Rate limiting considered
✓ HTTPS used

Data
✓ Sensitive data encrypted
✓ Access restricted

Monitoring
✓ Function logging enabled
✓ Audit logging enabled
✓ Security alerts configured
✓ Abnormal activity monitored

Incident Response
✓ Response procedures documented
✓ Logs retained appropriately
✓ Recovery/deployment mechanism available

LECTURE SUMMARY

Cloud-Native and Serverless Security

1 Cloud-Native and Serverless

Serverless computing allows developers to execute applications without directly managing the underlying servers.

2 Serverless Does Not Mean Security-Free

Customers remain responsible for code, IAM, data, secrets, dependencies, APIs, and configuration.

3 Major Security Principles

Least privilege, secure coding, input validation, strong auth, secret management, dependency security, encryption, continuous monitoring.

4 Monitoring

Cloud-native logs, metrics, audit records, and traces provide visibility into serverless environments.

5 Incident Response

Detect → Analyze → Contain → Eradicate → Recover → Improve.

6 DevSecOps Connection

Serverless security must be integrated into development, CI/CD, deployment, runtime monitoring, and incident response.

DEVSECOPS

Questions?